
How the Files Connect

A dependency graph and five flow diagrams for the Conversation feature — which file calls which, and what actually happens when a learner speaks.

Written for: a developer tracing this feature through the codebase

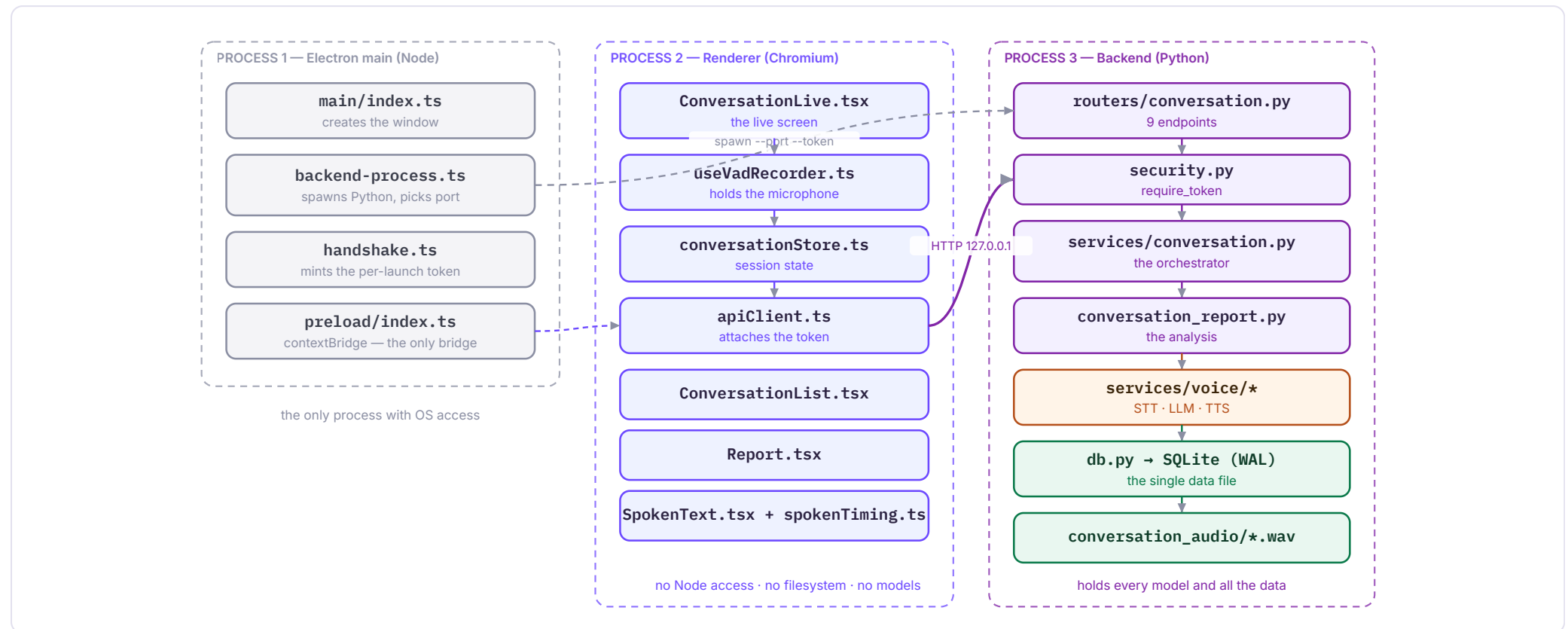
Edges: extracted from the real imports, not drawn from memory

Scope: 23 backend files · 11 renderer files · 53 import edges

System: FluencyOS · Electron + React + FastAPI + SQLite

1. Three processes, and the one channel between them

Every file in this document lives in exactly one of these three processes. That placement decides what it is allowed to do.

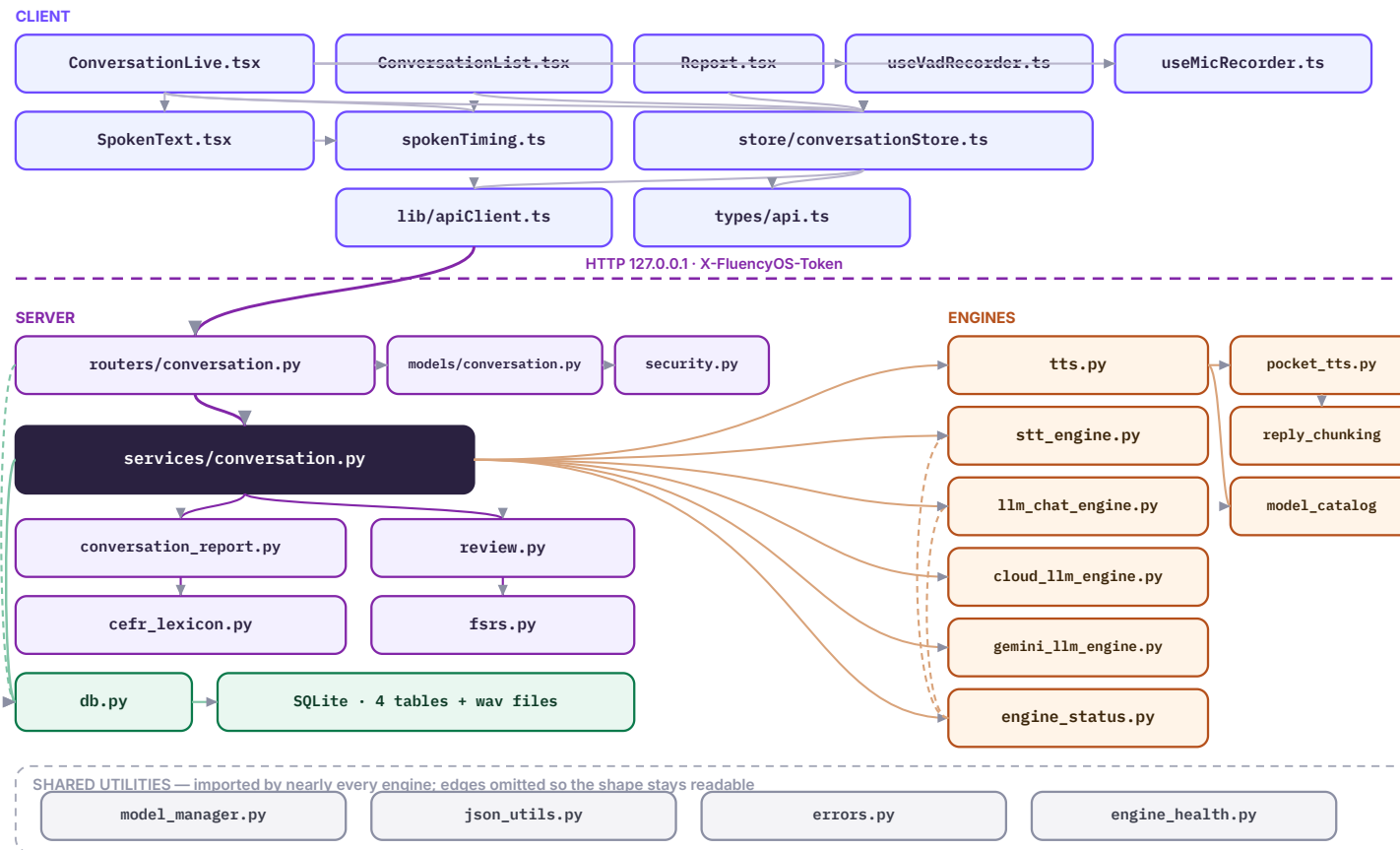


client (React) server (Python) engine (model) stored data shell / utility the orchestrator

The renderer holds nothing. No model, no database, no file handle. Everything it appears to do is an HTTP request to the backend, carrying a token minted by `handshake.ts` for this launch only. The backend rejects anything without it, so no other process on the machine can reach it even though the port is open locally.

2. The dependency graph

Read downward. An arrow means "imports and calls". All 53 edges were extracted from the source; the only ones omitted are those into the shared utilities band, which nearly every engine imports and which would otherwise obscure the shape.



The spine

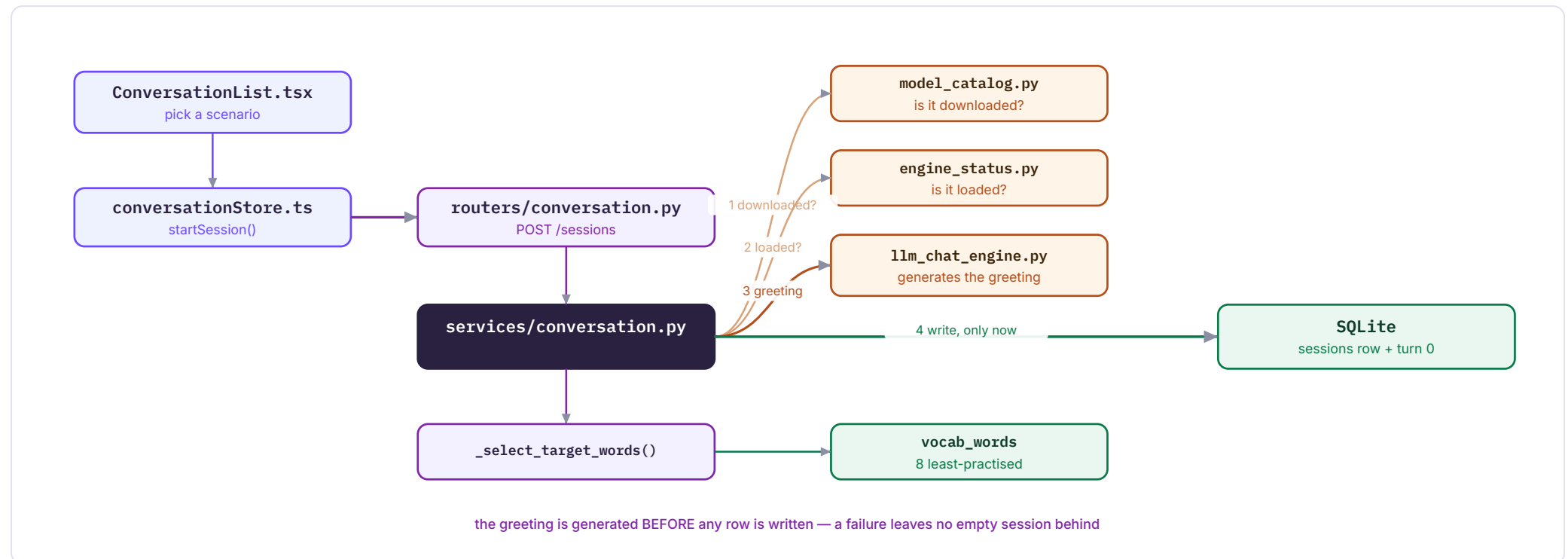
Nine files carry the feature. `services/conversation.py` is the waist of the hourglass: everything above it is transport, everything below it is capability, and it is the only file that talks to both.

What the graph tells you

No cycles, and the client never reaches past `apiClient.ts`. Adding an LLM provider touches three files; adding a scenario touches one. That is a property of this shape, not an accident.

3. Flow — starting a session

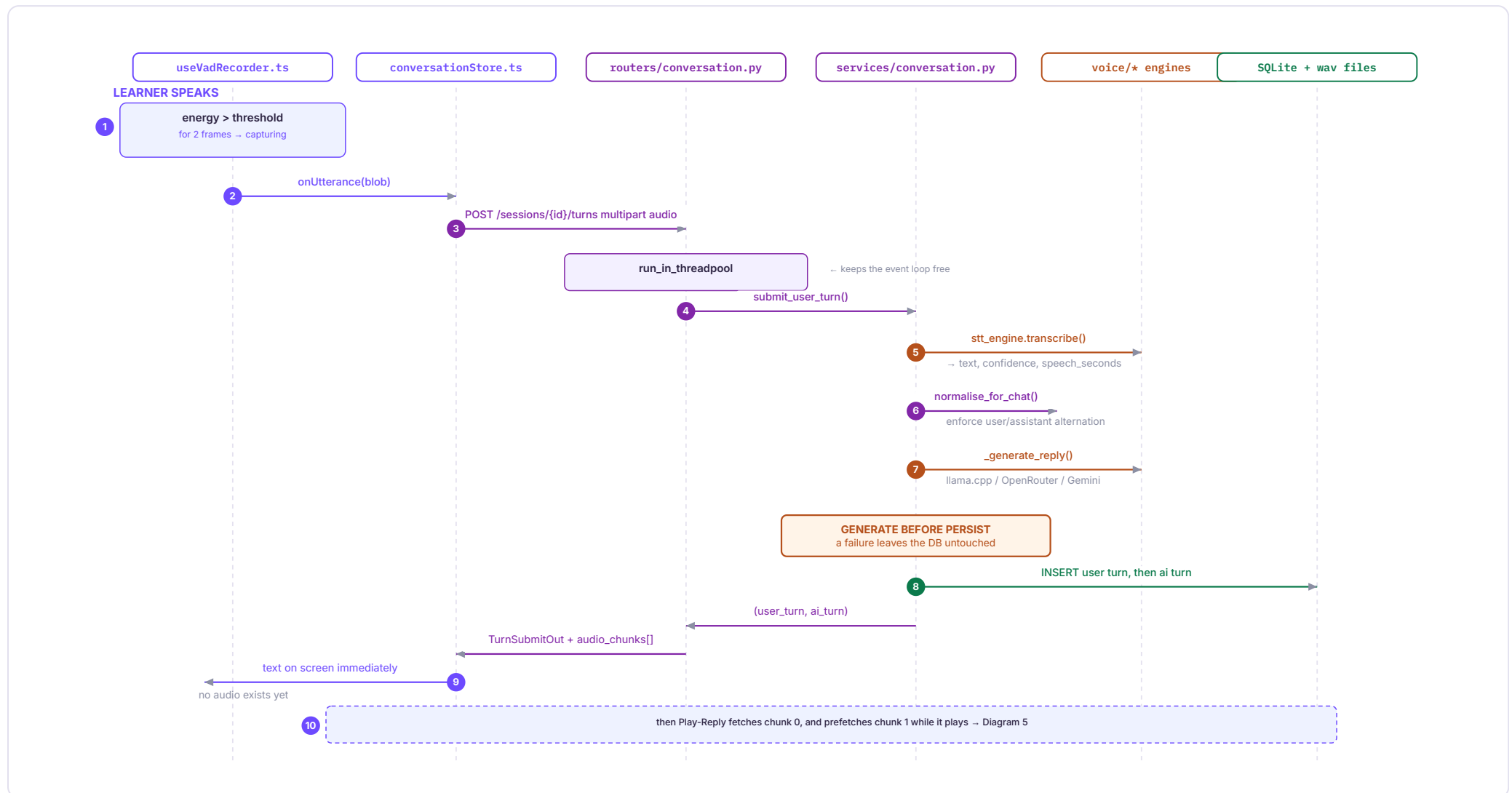
Four checks and one generation before a single row is written.



The ordering is the design. If the session row were written first and the greeting then failed, the learner's history would hold a conversation with no turns in it, forever — and "resume" could only ever reopen it empty. It also avoids holding a SQLite write transaction open across a call that can take minutes on a cold model load, which would block every other writer in the app.

4. Flow — one spoken turn

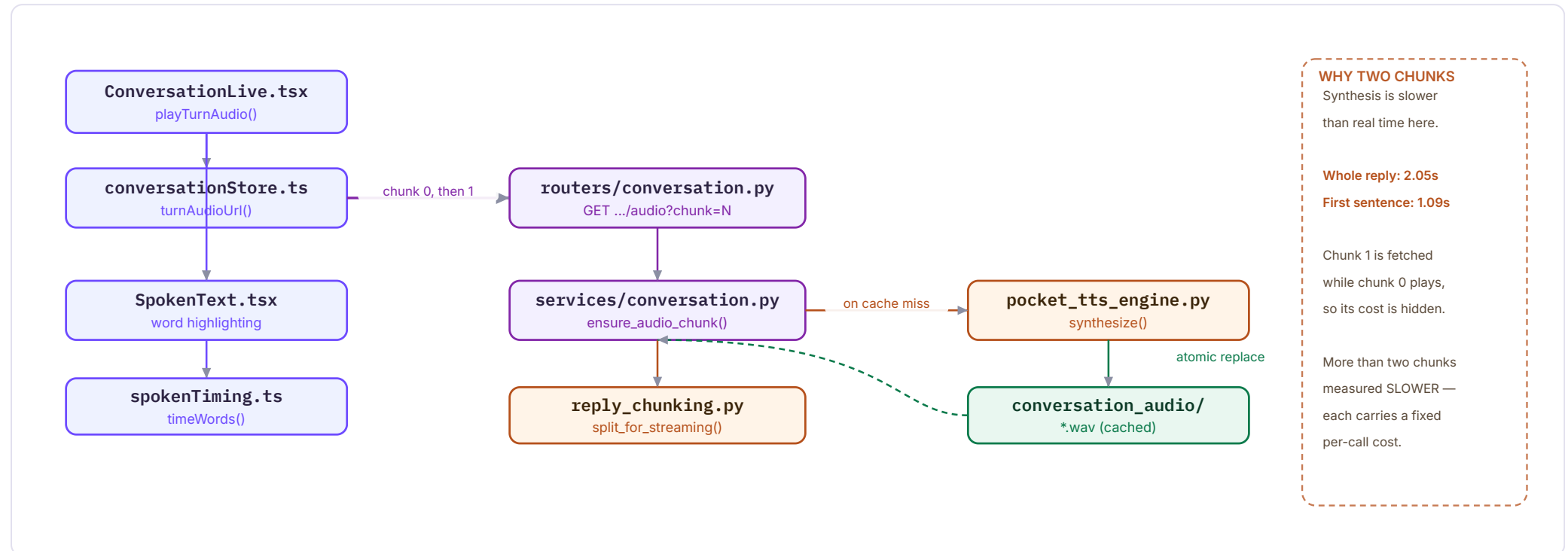
The central diagram. Time runs downward; each column is a file. This is the path taken once per exchange, many times per session.



Step 9 before step 10 is the most consequential ordering in the feature. The reply's text reaches the screen while no audio exists at all. Synthesis runs slower than real time on this hardware, so waiting for sound before showing words cost roughly 30 seconds per reply.

5. Flow — the audio, fetched separately

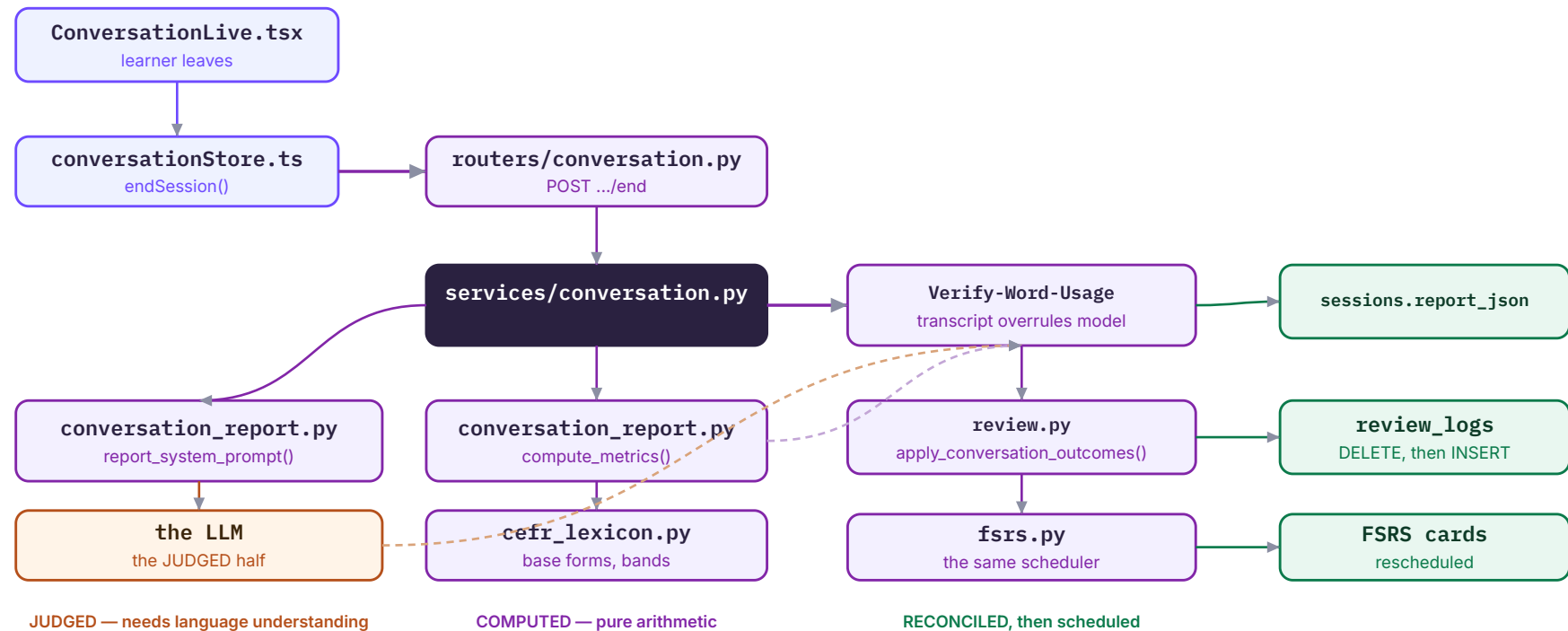
A second round trip that overlaps with the first. The learner is already reading while this runs.



Two requests for the same chunk genuinely overlap — a replay, a retry, a second window — because chunk 1 is fetched while chunk 0 is still audible. Writing straight to the final path would let a reader open a half-written WAV and serve it truncated, heard as a click or as silence. The file is written to a private temporary name and atomically renamed, so a reader sees either no file or the whole file.

6. Flow — ending a session

Two independent halves are computed, then reconciled at exactly one point, then applied to the scheduler.

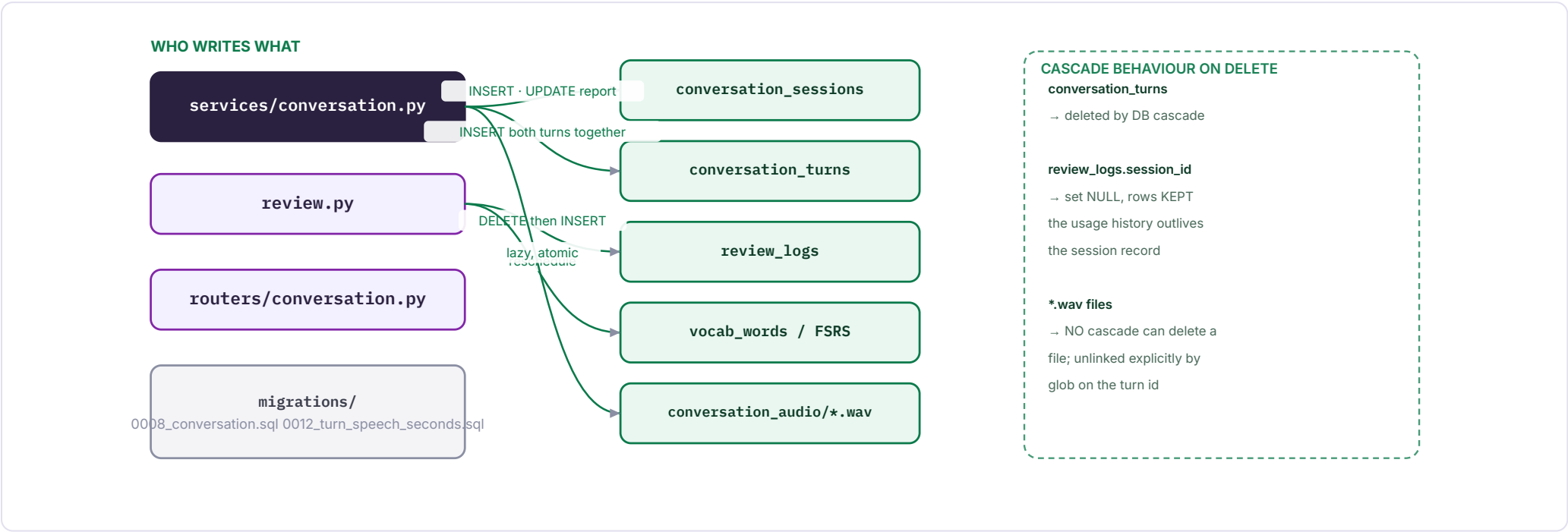


the two halves meet only at Verify-Word-Usage, which overrules the model exactly where the transcript disproves it

The judged and computed halves never mix. A model asked for a "fluency score" will invent one; a model asked whether a learner used a word correctly is doing work only a language model can do. Where the two disagree about whether a word was used at all, the transcript wins — that is a fact, not a judgement, and a model marking a word "avoided" that the learner plainly said would otherwise push it back to the front of the practice queue.

7. Where the data lands

Which file writes which table, and what survives a delete.



Tracing a change

To change...	Start at
What the AI says	services/conversation.py → _system_prompt
A report number	conversation_report.py
How audio is cut	reply_chunking.py
Mic sensitivity	useVadRecorder.ts
Scheduling effect	review.py

The three round trips

Trip	Endpoint	Cost
Start	POST /sessions	one generation
Each turn	POST .../turns	STT + generation
Each chunk	GET .../audio	synthesis, cached
End	POST .../end	one generation